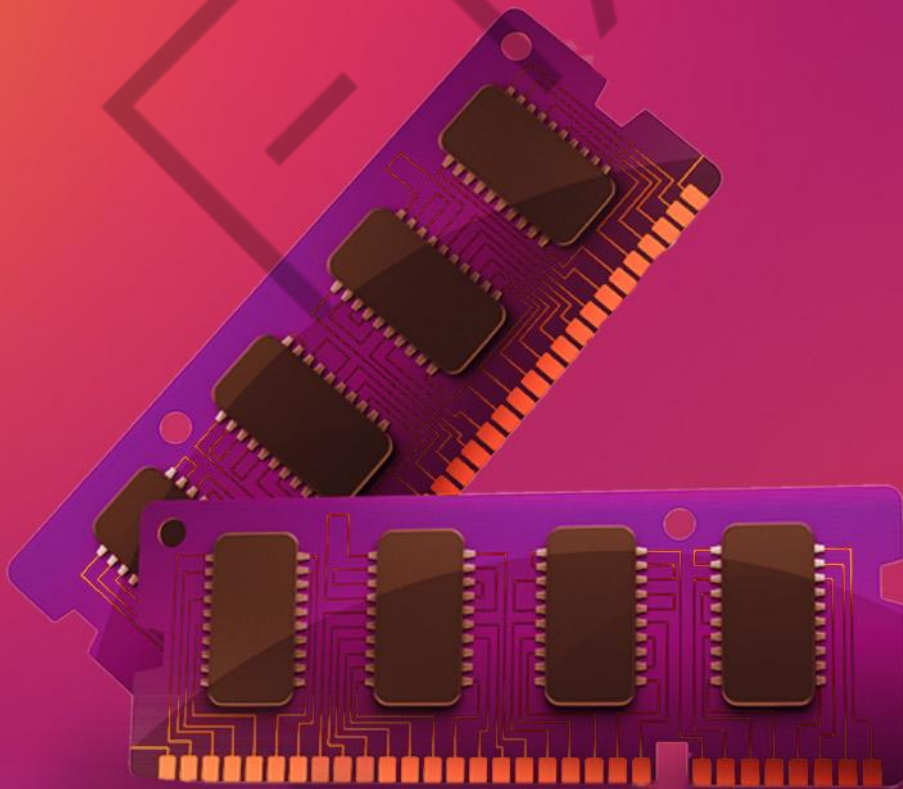


PARA O ALTO E AVANTE

# *PERÍCIA FORENSE EM SISTEMAS IN VIVO*



3

## LISTA DE FIGURAS

|                                                       |    |
|-------------------------------------------------------|----|
| Figura 1 – Abstração do modelo de Von Neumman.....    | 7  |
| Figura 2 – Saída do arquivo <i>/proc/iomem</i> .....  | 14 |
| Figura 3 – Módulo carregável pelo <i>kernel</i> ..... | 17 |
| Figura 4 – Conclusão do <i>dump</i> .....             | 18 |

EMANIP

## LISTA DE TABELAS

|                                                                                |   |
|--------------------------------------------------------------------------------|---|
| Tabela 1 – Degradação dos dados em função do tempo ( <i>sleep mode</i> ) ..... | 8 |
|--------------------------------------------------------------------------------|---|

EMSE

## LISTA DE QUADROS

|                                                    |    |
|----------------------------------------------------|----|
| Quadro 1 – Especificações da máquina virtual ..... | 16 |
|----------------------------------------------------|----|

EMSE

## LISTA DE CÓDIGO-FONTE

|                                                        |    |
|--------------------------------------------------------|----|
| Código-fonte 1 – Comandos para preparação do LKM ..... | 17 |
| Código-fonte 2 – Execução do <i>dump</i> .....         | 18 |
| Código-fonte 3 – Extração de <i>strings</i> .....      | 19 |
| Código-fonte 4 – Evidências obtidas .....              | 20 |

EMSE

## SUMÁRIO

|                                                                       |    |
|-----------------------------------------------------------------------|----|
| PERÍCIA FORENSE EM SISTEMAS IN VIVO .....                             | 7  |
| 1 INTRODUÇÃO .....                                                    | 7  |
| 2 PERÍCIA FORENSE EM SISTEMAS IN VIVO .....                           | 7  |
| 3 A RELEVÂNCIA DA ANÁLISE FORENSE DA RAM .....                        | 9  |
| 3.1 Processos.....                                                    | 9  |
| 3.2 Arquivos abertos e entradas de registro (“registry handles”)..... | 10 |
| 3.3 Arquivos em geral .....                                           | 10 |
| 3.4 Tráfego da rede .....                                             | 11 |
| 3.5 Dados da Internet.....                                            | 11 |
| 3.6 Senhas e chaves criptográficas.....                               | 12 |
| 3.7 Conteúdo decifrado .....                                          | 12 |
| 3.8 Outros dados que poderão ser encontrados na RAM .....             | 12 |
| 4 O DUMP DA MEMÓRIA VOLÁTIL EM SISTEMAS LINUX .....                   | 13 |
| 5 DESAFIOS RELACIONADOS À PRESERVAÇÃO DA RAM .....                    | 15 |
| 6 MÃOS À OBRA .....                                                   | 16 |
| 6.1 Hands-on: Dump da memória no Linux.....                           | 16 |
| REFERÊNCIAS.....                                                      | 21 |

## PERÍCIA FORENSE EM SISTEMAS IN VIVO

### 1 INTRODUÇÃO

A partir deste capítulo, você entenderá como ocorre a perícia em um dispositivo “*in vivo*” e qual o seu grau de importância. E se você já se perguntou se existe a perícia “*post-mortem*” em tom de brincadeira, saiba que existe sim, e que também a abordaremos nos capítulos posteriores, porém, por enquanto, tudo está bem vivo!

### 2 PERÍCIA FORENSE EM SISTEMAS IN VIVO

De acordo com Hausknecht, Foit e Buric (2015) apud Arruda (2018, p. 10):

[...] a análise *in vivo* passou a ser uma das novas armas do analista forense digital, haja vista que dados voláteis – dentre os quais destacam-se aqui os residentes na memória RAM ou dados não persistentes, como o conteúdo dos arquivos de hibernação ou de paginação da memória (*page file*), não poderão ser adequadamente preservados quando do desligamento do sistema sob investigação (objeto questionado), devendo-se observar, ainda, que significativa parcela dos dados e informações que trafega pelo objeto questionado, talvez nunca venha a ser registrada em disco, residindo unicamente na memória RAM.

Com base no modelo de Von Neumann (Figura “Abstração do modelo de Von Neumann”), pode-se afirmar que, em algum momento, todos os dados terão de passar pela RAM, nesse momento, serão gravados em blocos de memória, mesmo se não se relacionarem ao tráfego de rede ou qualquer outro fator.



Figura 1 – Abstração do modelo de Von Neumann  
Fonte: Wikipedia (2019)

Da Figura “Abstração do modelo de Von Neumman”, é possível observar que todos os dados manipulados pela CPU do sistema – cuja estrutura básica é simplificada representada na referida figura pelos registradores (acumulador), unidade de controle e unidade de aritmética e lógica – são continuamente gravados na RAM ou recuperados desta.

Farmer e Venema (2011), Hausknecht, Foit e Buric (2015) mencionam sobre a volatilidade dos dados, tendo em vista que os estados podem ser continuamente alterados no transcorrer do tempo, como consequência, o momento e a forma de execução da coleta de evidência importantes para o resultado da investigação.

A análise da Tabela “Degradação dos dados em função do tempo (*sleep mode*)” permite verificar a degradação (alteração) dos dados ao longo do tempo, estando os hosts de prova em *sleep mode* – aqui, simplificada definido como uma condição de baixo consumo de energia, durante a qual todas as ações no sistema são paralisadas, sendo os estados – inclusive de arquivos e aplicações abertas, gravados na memória RAM.

| Activity       | % Changes in RAM |            |
|----------------|------------------|------------|
|                | 256 MB RAM       | 512 MB RAM |
| Start          | 100              | 100        |
| Sleep mode 1h  | 90,4             | 96,7       |
| Sleep mode 2h  | 79,7             | 96,1       |
| Memory dump    | 76,9             | 89,8       |
| Sleep mode 15h | 74,8             | 85,6       |
| Memory dump    | 67,2             | 69,4       |

Tabela 1 – Degradação dos dados em função do tempo (*sleep mode*)  
Fonte: Hausknecht, Foit e Buric (2015)

Os dados da Tabela “Degradação dos dados em função do tempo (*sleep mode*)” foram obtidos por meio de dois experimentos executados com *hosts* antigos, providos de processadores AMD *single core* e sistemas operacionais Windows XP SP3. Num momento inicial, foi instalado um único módulo de memória 256MB/DDR1 em cada um desses *hosts*, sendo os procedimentos posteriormente refeitos com dois dos referidos módulos.



Após inicialização dos *hosts* de prova, há a execução das seguintes ações em cada um deles, a fim de originarem processos de controle:

- a. Abertura de um documento com o MS Office.
- b. Inicialização do IE e login em um serviço de e-mail on-line.
- c. Abertura de um terminal e, a partir desse, execução do comando *ipconfig* mais envio de um pacote ICMP (“ping”) a *google.com*.
- d. Abertura do Windows Explorer e execução de cópia de um arquivo.

Após o último passo (d), foi realizado um *dump* da RAM (256MB), sendo, então, um segundo teste com os mesmos *hosts*, desta vez equipados com um segundo módulo de memória (512MB).

### 3 A RELEVÂNCIA DA ANÁLISE FORENSE DA RAM

De acordo com Arruda (2018), algumas das informações mais relevantes possivelmente encontradas na análise da memória RAM são as que apresentaremos em seguida.

#### 3.1 Processos

Os processos ativos estão localizados na RAM e poderão ser recuperados pela sua localização. Além dos processos regulares, também há os ocultos como é o caso de códigos maliciosos (*malwares*) como *rootkits* – que poderão ser identificados com base na análise dos cabeçalhos/rodapés dos processos ativos, tarefa normalmente impossível de ser realizada por métodos tradicionais, por exemplo, por meio das ferramentas nativas do sistema operacional.

Sobre *rootkits*, Arruda (2018, p. 12) assim afirma:

Embora os *rootkits* geralmente tentem ocultar sua presença e suas ações no sistema subvertendo essas ferramentas, a análise dos blocos nos quais o processo se localiza poderá ajudar a recuperar informações inicialmente encobertas pelo código malicioso. Logo, pode-se observar que os procedimentos investigativos exigirão não apenas sólido conhecimento técnico por parte do investigador, bem como tempo hábil para adequada execução.

Cabe enfatizar que tais informações poderão ser recuperadas somente por meio da análise do conteúdo da memória RAM, ou seja, da análise *in vivo* do sistema alvo, e que tal abordagem é aplicável também a informações referentes a processos já finalizados, desde que o *host* não tenha sido reinicializado após o respectivo encerramento e que seus ambientes ainda não tenham sido atribuídos a outros processos, desta forma, persistindo na memória volátil, portanto, passíveis de recuperação.

### 3.2 Arquivos abertos e entradas de registro (“*registry handles*”)

Informações referentes a arquivos usados pelo sistema costumam ser especialmente relevantes. Sejam arquivos que acessem processos específicos ou mesmo entradas de registro, todos poderão ser encontrados na RAM.

Caso um processo seja parte de um *malware*, os arquivos por ele abertos poderão fornecer ao investigador informações sobre o local onde seu código reside no disco rígido, para o qual, possivelmente, os dados estão sendo enviados, e ainda, arquivos que eventualmente tenham sido modificados, dentre outras informações. A análise de tais vestígios pode contribuir para a determinação da autoria e do objetivo do malware, permitindo, também, identificar as vulnerabilidades por ele exploradas, contribuindo, dessa forma, para os devidos ajustes de segurança do sistema.

### 3.3 Arquivos em geral

Do anteriormente exposto, é plausível afirmar que todos os arquivos de alguma forma manipulados pelo sistema, necessariamente, passam pela RAM. Por exemplo, arquivos abertos para leitura ou para edição. Considerando-se também que, normalmente, enquanto um arquivo encontra-se em edição, o *software* cria uma cópia temporária desse arquivo, a qual permanece residente na RAM até que o processo de edição seja finalizado e as modificações gravadas em disco, é possível que essa cópia temporária do arquivo ou fragmentos dela possam ser recuperados quando da análise da RAM. Cabe ressaltar que quanto maior a quantidade de memória RAM disponível, potencialmente maior será a quantidade de arquivos de alguma forma manipulados pelo sistema.

Tendo em vista que algumas das operações mais comumente executadas com arquivos recaem sobre um simples *cut & paste* (recortar e colar) de blocos de informações e, ainda, que o conteúdo desses blocos é temporariamente armazenado na área de transferência (*clipboard*), pode-se concluir que, ao possibilitar a recuperação de tais informações, a análise do conteúdo da RAM mais uma vez se apresenta como valioso recurso para o esclarecimento e resposta a incidentes.

### 3.4 Tráfego da rede

A análise do conteúdo da RAM poderá revelar conexões estabelecidas e portas abertas no objeto questionado, ajudando a verificar, dentre outras possibilidades, a existência de *backdoors*. Vale dizer que a extração de tais informações diretamente da RAM aumenta consideravelmente a credibilidade da evidência digital, na medida em que impede a ocultação de eventuais conexões ilegítimas abertas por processos maliciosos.

### 3.5 Dados da Internet

Dada a popularização do uso da Internet, desde a simples navegação por diversão até o acesso aos mais variados tipos de serviços para fins pessoais ou profissionais, pode-se afirmar que também os dados obtidos de tais atividades poderão ser consideravelmente relevantes. Hausknecht, Foit e Buric (2015) destacam a preferência dos usuários pelo armazenamento de seus arquivos em nuvem, em detrimento do armazenamento local. Destaca-se também o uso de serviços on-line, como os serviços de correio eletrônico do GMAIL e Yahoo, e os serviços de telecomunicações como o Skype. Segundo esses autores, todos os dados baixados da Internet sempre passarão pela RAM, possibilitando que todas as mensagens, os arquivos, as conversas e outras informações sejam preservados por meio da imagem física (*dump*) da RAM para posterior recuperação e análise.

### 3.6 Senhas e chaves criptográficas

Certamente, um dos pontos de maior reconhecimento da análise forense *in vivo* recai sobre a possibilidade de recuperação de senhas e chaves criptográficas. Tais informações raramente são gravadas em disco sem proteção adicional, entretanto, quando em uso, precisam ser armazenadas na RAM, lá permanecendo residentes até que sejam sobrescritas por outros dados ou até que a máquina seja desligada, o que permite que sejam capturadas pelo processo de *dump*.

### 3.7 Conteúdo decriptografado

Frequentemente, a busca por senhas e chaves criptográficas na RAM leva os investigadores também a dados decriptografados, os quais podem ser recuperados. Ao se decriptografar um arquivo, ele será temporariamente armazenado na RAM, assim sendo, mais uma vez, a respectiva análise do conteúdo permitirá encontrar fragmentos de documentos ou até documentos inteiros que tenham sido anteriormente criptografados, diferentemente da análise “*post-mortem*”, na qual será possível encontrar apenas arquivos ainda criptografados.

### 3.8 Outros dados que poderão ser encontrados na RAM

Além dos anteriormente mencionados, outros dados poderão ser obtidos por meio da análise do conteúdo da RAM, como os *usernames* em utilização no sistema, a hora de *logon* dos usuários e as ações por eles executadas durante sua sessão. Ainda de acordo com Hausknecht, Foit e Buric (2015), isso é aplicável a diferentes sistemas operacionais, como o *Microsoft Windows*, *Apple OSX* e várias distribuições Linux, entretanto, dadas suas particularidades, nem todos os sistemas operacionais apresentarão os mesmos resultados. Todavia, informações valiosas como conexões de rede, processos e até arquivos, dentre outras, poderão ser recuperadas a partir do *dump* da RAM de qualquer um deles, observando-se, por fim, que em ambientes virtualizados, o *dump* da RAM (virtualizada) poderá ser bastante simplificado, uma vez que, normalmente, os dados nela residentes são armazenados como um arquivo único no disco rígido do hospedeiro.

## 4 O *DUMP* DA MEMÓRIA VOLÁTIL EM SISTEMAS LINUX

Segundo Kollár (2010), apud Arruda (2018) módulos carregáveis pelo *kernel* ou simplesmente LKMs (*loadable kernel modules*) podem ser utilizados para *dump* do conteúdo da memória volátil (RAM) em sistemas Linux, quando adequadamente preparados.

Diferentemente de outros sistemas operacionais, como *Windows* e *Mac OS*, o código do *kernel* Linux é aberto e pode ser livremente personalizado pelo usuário ou pelos responsáveis pela distribuição. Logo, diferentes técnicas para aquisição do conteúdo da RAM podem ser aplicadas a sistemas Linux, algumas em *user mode*, outras em *kernel mode*.

Segundo Hale–Ligh *et al.* (2014), o `/dev/mem` era a interface (*user mode*) mais utilizada para aquisição do conteúdo da RAM, antes de ser desabilitado na maior parte das distribuições Linux por motivos de segurança. Consistindo em um arquivo de dispositivo tipo caractere, que é uma imagem da memória física do computador, foi originalmente idealizado para possibilitar o exame e a aplicação de correções do sistema. Quando habilitado, o `/dev/mem` permite que ferramentas como o *dd* possam ler ou escrever diretamente na RAM, sendo os endereços de *bytes* no `/dev/mem` interpretados como endereços de memória física e, assim, eventuais referências a locais inexistentes causarão erros.

Uma das limitações do `/dev/mem` recai sobre sua capacidade de endereçar apenas os primeiros 896MB de RAM, mesmo que a capacidade de memória do sistema seja superior a isso, devendo-se considerar, ainda, que muitas máquinas não mapeiam a RAM contiguamente a partir do *offset* físico 0. Com isso, um investigador inexperiente poderia acessar regiões sensíveis, causando a corrupção da memória ou instabilidade geral do sistema.

Em seu trabalho, Kollár (2010) reitera que uma das maneiras mais práticas para execução do *dump* da memória é por intermédio da ferramenta *dd*, executada com privilégios de *root*. Entretanto, frente às limitações do `/dev/mem`, o autor concluiu que uma solução mais adequada seria a criação de um módulo (LKM) especificamente destinado a tal finalidade, o qual, analogamente ao `/dev/mem`, também criasse um dispositivo de caractere virtual, porém, livre das suas limitações.

Em sistemas Linux, o arquivo `/proc/iomem` mostra o mapeamento atual da memória para cada dispositivo físico. A Figura “Saída do arquivo `/proc/iomem`” oferece um exemplo da saída desse arquivo, com base em um *host* de testes executando a distribuição Linux *Debian*, com 1536MB de RAM, virtualizado em ambiente *Oracle VirtualBox*.

```
user1@target:~$ cat /proc/iomem
00000000 - 00000fff : reserved
00001000 - 0009fbff : System RAM
0009fc00 - 0009ffff : reserved
000a0000 - 000bffff : PCI Bus 0000 : 00
000c0000 - 000c7fff : Video ROM
000e2000 - 000e2fff : Adapter ROM
000f0000 - 000fffff : reserved
000f0000 - 000fffff : System ROM
00100000 - 5ffeffff : System RAM
01000000 - 01520070 : Kernel code
01520071 - 018ecfbf : Kernel data
01a22000 - 01af3fff : Kernel bss
5fff0000 - 5fffffff : ACPI Tables
60000000 - ffdfffff : PCI Bus 0000 : 00
e0000000 - e0ffffff : 0000 : 00 : 02 : 0
f0000000 - f001ffff : 0000 : 00 : 03 : 0
f0000000 - f001ffff : e1000
f0400000 - f07fffff : 0000 : 00 : 04 : 0
f0400000 - f07fffff : vboxguest
f0800000 - f0803fff : 0000 : 00 : 04 : 0
f08040000 - f0805fff : 0000 : 00 : 06 : 0
f0804000 - f0804fff : ohci_hcd
f0805000 - f0805fff : 0000 : 00 : 0b : 0
f0805000 - f0805fff : ehci_hcd
f0806000 - f0807fff : 0000 : 00 : 0d : 0
f0806000 - f0807fff : ahci
fec00000 - fec00fff : reserved
fec00000 - fec003ff : IOAPIC 0
fee00000 - fee00fff : Local APIC
fee00000 - fee00fff : reserved
ffc00000 - ffffffff : reserved
```

Figura 2 – Saída do arquivo `/proc/iomem`  
Fonte: Elaborado pelo autor

Os destaques da Figura “Saída do arquivo `/proc/iomem`” mostram que o arquivo `/proc/iomem` aponta que dois segmentos de memória (00000000-00000fff e 0009fc00-0009ffff) se encontram reservados, e que os segmentos cujos conteúdos deverão ser preservados são apontados pelo identificador System RAM (00001000-0009fbff e 00100000-5ffeffff).

A imagem forense pode ser simplificada como uma cópia (duplicação) *bit stream* (*bit a bit*) do conteúdo original desejado, podendo esse conteúdo consistir em um arquivo, partição simples, um disco inteiro ou mesmo o conteúdo da memória RAM do objeto questionado, dentre outras possibilidades. O *dd*, já anteriormente referido, é uma ferramenta *Unix/Unix-Like* simples, certificada e apta à produção de imagens forenses em formato *raw*.

## 5 DESAFIOS RELACIONADOS À PRESERVAÇÃO DA RAM

Do exposto, é possível afirmar que, em última análise, a preservação do conteúdo da RAM será concretizada sob a forma de seu *dump*.

Entretanto, o dinamismo e a volatilidade, duas das principais características desse tipo de memória, representam grandes desafios ao processo de preservação, na medida em que, do Princípio da Incerteza de Heisenberg, é sabido ser impossível a obtenção da totalidade dos dados residentes na RAM, em razão dos diferentes estados que as múltiplas regiões dessa memória poderão assumir ao longo do processo, o que torna imprescindível, portanto, a rigorosa observância da Ordem de Volatilidade (FARMER; VENEMA, 2011) a fim de maximizar o volume de dados preservados e coletados.

Considerando-se também a eventual necessidade de novas perícias do objeto questionado (adaptado do CPP - Art. 170), especificamente em regime *post-mortem*, a necessidade de preservação do ambiente como um todo emerge, embora tangencialmente ao contexto estudado, como um novo desafio para o analista forense, traduzido essencialmente no local de armazenamento do *dump*. De acordo com Jones e Etzkorn (2016), as opções mais comumente adotadas para esse propósito recaem:

- sobre o disco rígido do objeto questionado;
- sobre mídias USB removíveis – especialmente *pen drives* e discos rígidos externos;
- sobre sistemas de armazenamento em nuvem;
- sobre unidades de armazenamento remotas disponíveis via rede local – opção apontada por esses autores como a mais adequada.

Outro relevante aspecto a ser considerado pelo analista forense refere-se ao fato de que o sistema operacional deve proteger os diversos processos em execução em um sistema multiprogramação, inclusive, impedindo que o espaço de endereçamento de um processo venha a ser acessado por outro (OLIVEIRA; CARISSIMI; TOSCANI, 2010), o que significa que, em princípio, o sistema operacional em execução no objeto questionado não permitirá a realização do *dump* da RAM, conforme este necessita de acesso a regiões da memória já alocadas a outros processos.

## 6 MÃOS À OBRA

### 6.1 Hands-on: Dump da memória no Linux

Nesta primeira abordagem prática, será executado o *dump* do conteúdo da RAM de uma máquina virtual (VM), cujas especificações são descritas no Quadro “Especificações da máquina virtual”.

| Parâmetro           | Especificação     |
|---------------------|-------------------|
| Hypervisor          | VirtualBox        |
| Sistema operacional | Debian 9 – 64bits |
| Memória             | 1GB               |
| Disco               | 8GB               |
| Rede                | Bridge            |
| vCPU                | 1                 |

Quadro 1 – Especificações da máquina virtual

Fonte: Elaborado pelo autor (2019)

Para execução do *dump*, será utilizado o LiME – *Linux Memory Extractor*, um módulo carregável pelo *kernel* (LKM) que permite a aquisição do conteúdo da memória volátil de dispositivos Linux e *Linux-based*, como o Android. O LiME minimiza a interação entre processos em *user space* e *kernel space* durante a fase de aquisição, produzindo imagens mais alinhadas com as necessidades forenses. Embora o LiME possa fazer a aquisição do conteúdo da RAM via rede, nesta primeira incursão, o *dump* da RAM será gravado diretamente do disco rígido do objeto questionado.

É importante salientar que, em situação real, a instalação dos pacotes adicionais necessários para compilação do LKM, o procedimento de compilação e a gravação do *dump* no próprio disco rígido do objeto questionado implicariam em contaminação, geralmente, inaceitável desse. Tal contaminação poderia impactar consideravelmente sobre a preservação da evidência digital residente na RAM (necessária a análise *in vivo*), e residente também no disco rígido do objeto questionado (análise *post mortem*).

Os seguintes pacotes adicionais são necessários ao *download* e compilação do LiME:

- *Git*: pacote para *download* do LiME a partir do GitHub.



- *Build-essential*: contém uma lista informativa de pacotes que são considerados essenciais para a construção de (outros) pacotes no Debian.
- *Linux-headers*: arquivo de cabeçalhos necessários, dentre outras coisas, à compilação do LKM (LiME).

A partir de um terminal privilegiado (*root*) na VM, a instalação dos pacotes e a preparação do LiME poderão ser feitas por meio das linhas apresentadas no Código Fonte “Comandos para preparação do LKM”.

```
#atualiza a lista de pacotes que podem ser instalados no sistema, e se
nenhum erro ocorrer (&&), instala os pacotes especificados
root@deb00:~# apt-get update && apt-get install -y git build-
essential linux-headers-$(uname -r)

#baixa o LiME diretamente do GitHub, criando seu diretório de trabalho
root@deb00:~# git clone https://www.github.com/504ensicsLabs/Lime

#entra do diretório onde residem os arquivos para compilação do LiME
root@deb00:~# cd Lime/src

#compila o LiME conforme descrito pelo Makefile
root@deb00:~/Lime/src# make
```

Código-fonte 1 – Comandos para preparação do LKM  
Fonte: Elaborado pelo autor (2019)

Se compilado com sucesso, o LKM (arquivo com extensão .ko) já estará pronto para ser carregado para a memória (Figura “Módulo carregável pelo *kernel*”).

```
root@deb00:~/Lime/src# make
make -C /lib/modules/4.9.0-7-amd64/build M="/root/Lime/src" modules
make[1]: Entering directory '/usr/src/linux-headers-4.9.0-7-amd64'
CC [M] /root/Lime/src/tcp.o
CC [M] /root/Lime/src/disk.o
CC [M] /root/Lime/src/main.o
CC [M] /root/Lime/src/hash.o
LD [M] /root/Lime/src/lime.o
Building modules, stage 2.
MODPOST 1 modules
CC      CC
LD [M]  LD [M]
make[1]: Leaving directory '/usr/src/linux-headers-4.9.0-7-amd64'
strip --strip-unneeded lime.ko
mv lime.ko lime-4.9.0-7-amd64.ko
root@deb00:~/Lime/src# █
```

Figura 3 – Módulo carregável pelo *kernel*  
Fonte: Elaborado pelo autor (2019)

Vale salientar que este LKM poderá ser utilizado em qualquer outro sistema Linux que execute exatamente o mesmo *kernel* sobre o qual ele foi compilado (plataforma amd64, versão 4.9.0-7, e no caso, *kernel* original sem nenhuma personalização). O *dump* poderá ser iniciado com esta linha de comando (Código-fonte “Execução do *dump*”).

```
root@deb00:~/Lime/src# insmod lime-4.9.0-7-amd64.ko
path=dump.lime format=lime digest=sha256
```

Código-fonte 2 – Execução do *dump*  
 Fonte: Elaborado pelo autor (2019)

Da linha de comando na Código-fonte “Execução do *dump*”, tem-se:

- **insmod lime-4.9.0-7-amd64.ko:** carregamento do LKM para a RAM por intermédio do comando `insmod`.
- **format=lime:** o arquivo de *dump* será gerado em formato lime.
- **path=dump.lime:** cria um *output file* (arquivo de *dump*) chamado `dump.lime`, no diretório corrente.
- **digest=sha256:** calcular automaticamente o *hash* sha256 do arquivo de *dump*.

Tratando-se de um trabalho forense, é imprescindível que a integridade do arquivo de *dump* produzido pela ferramenta seja atestada mediante a extração do respectivo *hash*, embora não obrigatoriamente, já na linha de comando de execução do LiME.

Observe-se ainda que, tratando-se de imagens forenses (*dumps*), recomenda-se enfaticamente a extração de dois *hashes* diferentes, conforme a Figura “Conclusão do *dump*”.

```
root@deb00:~/Lime/src# insmod lime-4.9.0-7-amd64.ko path=dump.lime format=lime digest=s
ha256
root@deb00:~/Lime/src# sha1sum dump.lime | tee dump.lime.sha1 [1]
80076c4cbb798aae82802664a9e0340411957447 dump.lime
root@deb00:~/Lime/src# ls -lh dump.lime*
-r--r--r-- 1 root root 1,0G jan 16 11:00 dump.lime
-rw-r--r-- 1 root root 52 jan 16 11:00 dump.lime.sha1 [2]
-r--r--r-- 1 root root 64 jan 16 11:00 dump.lime.sha256
root@deb00:~/Lime/src# █
```

Figura 4 – Conclusão do *dump*  
 Fonte: Elaborado pelo autor (2019)

No destaque 1 da figura “Conclusão do *dump*”, o comando `sha1sum` calcula o *hash* sha1 do arquivo `dump.lime` e o imprime na tela, e ao mesmo tempo também o grava no arquivo `dump.lime.sha1`.

No destaque 2 da mesma figura, podem ser observados o arquivo de *dump* (`dump.lime`) e seus respectivos *hashes*.

A exploração inicial do conteúdo do *dump* pode ser feita mediante a extração das *strings* nele existentes, possibilitando-se, assim, a busca de evidências (Código-fonte “Extração de *strings*”).

```
root@deb00:~/Lime/src# strings -a dump.lime > dump.str
```

Código-fonte 3 – Extração de *strings*

Fonte: Elaborado pelo autor (2019)

A linha de comando no Código-fonte “Extração de *strings*” extrai as *strings* do arquivo *dump.lime*, redirecionando-as ao arquivo *dump.str* (criado *on the fly* por meio do redirecionador *>*). Com essas *strings*, é possível obter-se várias evidências, por exemplo, verificar que (Código-fonte “Evidências Obtidas”):

- O *host* foi configurado automaticamente, via DHCP.
- Seu endereço IPv4 é 192.168.33.16.
- Seu endereço IPv6 é 2804:431:d719:cecf:a00:27ff:fe67:8813.
- A *interface* de rede *eth0* foi renomeada para *enp0s3*.
- A linha de comando utilizada para inserção do LiME.
- O *hostname* do objeto questionado é *deb00*.

```
root@deb00:~/Lime/src# grep enp0s3 dump.str
MESSAGE=Joining mDNS multicast group on interface enp0s3.IPv6
with address 2804:431:d719:cecf:a00:27ff:fe67:8813. [c]
MESSAGE=New relevant interface enp0s3.IPv6 for mDNS.
MESSAGE=Joining mDNS multicast group on interface enp0s3.IPv4
with address 192.168.33.16. [b]
! <-- muitas linhas omitidas -->
MESSAGE=DHCPDISCOVER on enp0s3 to 255.255.255.255 port 67
interval 6
MESSAGE=DHCPREQUEST [a] of 192.168.33.16 on enp0s3 to
255.255.255.255 port 67
MESSAGE=enp0s3=enp0s3
MESSAGE=e1000 0000:00:03.0 enp0s3: renamed from eth0 [d]
! <-- muitas linhas omitidas -->
!
root@deb00:~/Lime/src# grep lime dump.str
! <-- muitas linhas omitidas -->
insmod lime-4.9.0-7-amd64.ko path=dump.lime format=lime
digest=sha256 [e]
!<-- muitas linhas omitidas -->
!
root@deb00:~/Lime/src# grep hostname dump.str
! <-- muitas linhas omitidas -->
```

```
MESSAGE=Set hostname to <deb00>. [f]  
! <-- muitas linhas omitidas -->  
!
```

Código-fonte 4 – Evidências obtidas  
Fonte: Elaborado pelo autor (2019)

Embora tais informações possam, inicialmente, parecer triviais, há que se considerar que são mais confiáveis do que se obtidas diretamente a partir das ferramentas do sistema operacional de um sistema sob suspeita de ataque, uma vez que tais ferramentas podem vir a ser subvertidas, por exemplo, por códigos maliciosos. Pode-se concluir, então, que a análise forense do conteúdo da RAM constitui ferramenta de grande valia também quando da resposta a incidentes de segurança.

E não acaba por aqui. Ainda temos a segunda parte da perícia “in vivo”. Até lá!

## REFERÊNCIAS

ARRUDA, O. D. R. **Uma proposta para automatização do processo de preservação de evidências voláteis em sistemas *in vivo* com ênfase em hosts Linux**. 2018. Dissertação (Mestrado em Engenharia da Computação) – Instituto de Pesquisas Tecnológicas do Estado de São Paulo, São Paulo.

CASTRO JÚNIOR, A. P.; LISITA, B. L.; MOURA, T. S. M.; PINTO, T. J. **Forense Computacional em Memória Principal**. 2012. Disponível em: <<https://pt.scribd.com/document/517417003/Forense-Computacional-Em-Memoria-Principal>>. Acesso em: 28 maio 2021.

FARMER, D.; VENEMA, W. **Perícia Forense Computacional**. Teoria e Prática Aplicada. 4. Reimpressão. São Paulo: Pearson Prentice Hall, 2011.

HALE-LIGH, M.; CASE, A.; LEVY, J.; WALTERS, A. **The Art of Memory Forensics**. Detecting Malware and Threats in Windows, Linux, and Mac Memory. Indianapolis: John Wiley & Sons Inc., 2014.

HAUSKNECHT, K.; FOIT, D.; BURIC, J. RAM data significance in digital forensics. In: **38th International Convention on Information and Communication Technology, Electronics and Microelectronics**, MIPRO 2015. Opatija, Croatia, May 2015.

HORSWELL, J. **The Practice of Crime Scene Investigation**. New York: CRC Press, 2004.

JONES, J.; ETZKORN, L. Analysis of Digital Forensics Live System Acquisition Methods to Achieve Optimal Evidence Preservation. **SoutheastCon**, p. 1-6, 2016.

KOLLÁR, I. **Forensic RAM dump image analyser**. 2010, 61f. Master Thesis - Charles University in Prague Faculty of Mathematics and Physics, Prague, 2010.

LAZARIDIS, I.; ARAMPATZIS, T. POURROS, S. **Evaluation of Digital Forensics Tools on Data Recovery and Analysis**. Proceedings of the Third International Conference on Weblogs and Social Media, Thessaloniki, p. 67-71, may, 2009.

OLIVEIRA, R. S.; CARISSIMI, A. S.; TOSCANI, S. S. **Sistemas Operacionais**. 4. ed. Porto Alegre: Bookman, 2010.

VANDEVEN, S. **Forensic Images: for your viewing pleasure**. 2014. Disponível em: <https://www.sans.org/reading-room/whitepapers/forensics/forensic-images-viewing-pleasure-35447>. Acesso em: 28 maio 2021.